CLASE 1 · 3 HS

Hola ESP32

Escribir, correr y modificar los primeros programas en C. Al salir, un LED parpadea porque **vos** se lo ordenaste.

Programación en C con ESP32 · Sistemas Lógicos · Universidad Nacional de Santiago del Estero · 2026

HOY

Seis bloques, un LED que parpadea

1 · 20 min

¿Qué es programar?

2 · 20 min

PlatformIO y Wokwi

3 · 30 min

El primer programa

4 · 40 min

Encender un LED

5 · 20 min

Leer errores

6 · 50 min

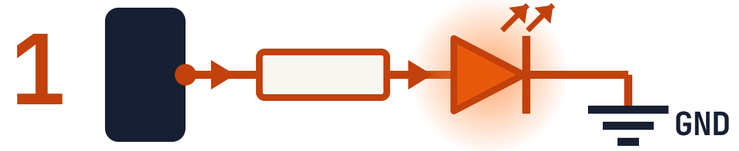
Ejercicios y TP

Un microcontrolador es una computadora en un chip

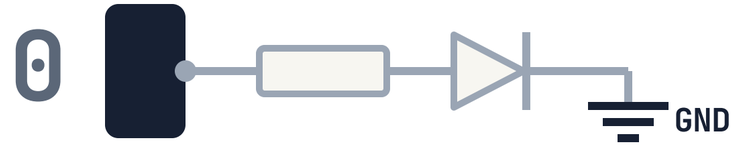
No tiene pantalla ni teclado. Tiene **pines**: patitas que ponen o leen una tensión.

Un **programa** es una lista de instrucciones que el micro ejecuta **de a una, en orden, de arriba hacia abajo**.

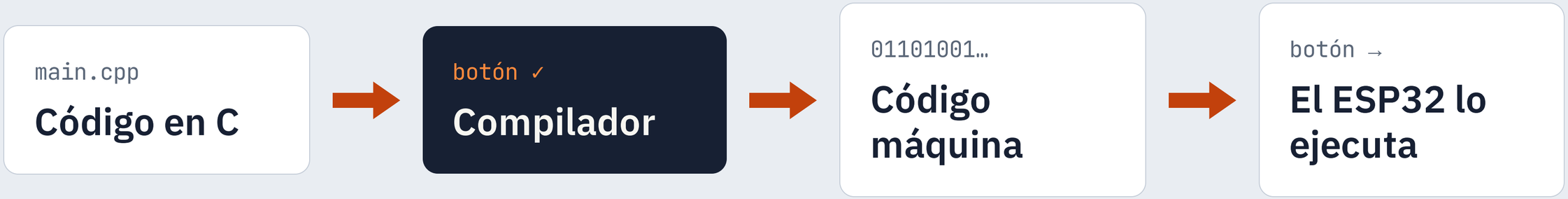
Salida en **3,3 V**: el LED enciende



Salida en **0 V**: el LED apaga



El micro no entiende C: el compilador lo traduce



Si el compilador no entiende algo, **no traduce nada** y muestra un **error**. Es normal: los programadores con años de experiencia ven errores todos los días.

El mismo código, dos formas de probarlo

VS Code + PlatformIO

En la PC. El mismo entorno de Microprogramables.

- *File → Open Folder…* → la carpeta del proyecto
- ✓ compilar · → cargar · 🔌 monitor
- Cargar en la placa: DevKit o C3

Wokwi en el navegador

Sin instalar nada y sin placa: facultad, casa o celular.

- Armar la placa de cátedra (pegar **diagram.json**)
- Programas listos en la carpeta **wokwi/**
- ▶ para correr

TP de hoy: copiar **platformio/plantilla/** y renombrarla **SL2026-Apellido-Clase1**.

Dos placas, un solo programa

Elemento	ESP32 DevKit	ESP32-C3 Super Mini
LEDs L0 … L7	4, 16, 17, 18, 19, 21, 22, 23	0, 1, 3, 4, 5, 6, 7, 10
Pulsadores A, B, C	32, 33, 25	20, 21, 9 (BOOT)
Entorno en PlatformIO	env:esp32dev	env:esp32c3



El código se escribe **siempre con los pines del DevKit**. En el C3, PlatformIO los traduce solo. Lo único que cambia es el entorno elegido en la barra azul.

Hola, mundo

```
#include <Arduino.h>

void setup() {
  Serial.begin(115200);
  Serial.printf("Hola, mundo!\n");
}

void loop() {
}
```

Escribirlo en **src/main.cpp** (o en **sketch.ino** de Wokwi), cargarlo y abrir el monitor serie.

```
Hola, mundo!
```

#include <Arduino.h> avisa que vamos a usar las funciones del ESP32. Va siempre, en la primera línea.



Todo programa tiene dos bloques

`setup()`

Se ejecuta **UNA vez**, al arrancar.

Configurar pines, abrir el monitor serie, imprimir un cartel.

`loop()`

Se repite **PARA SIEMPRE.**

Lo que el micro hace todo el tiempo: parpadear, leer botones, contar.

encender → `setup()` → `loop()` → `loop()` → … hasta que se apague

Las reglas de escritura

Regla	Ejemplo
Cada instrucción termina con **punto y coma**	`delay(500);`
Los bloques van entre **llaves**	`void loop() { … }`
C distingue **mayúsculas y minúsculas**	`Serial ≠ serial`
El texto va entre **comillas dobles**	`"Hola"`
// empieza un comentario: el compilador lo ignora	`// para humanos`
Los espacios no importan, pero **la sangría ayuda a leer**	ver los ejemplos

Imprimir en el monitor serie

`Serial.begin(115200);`	Abre la comunicación con la PC a 115200 baudios. Va **una vez**, en setup().
`Serial.printf("texto\n");`	Imprime el texto en el monitor serie.
`\n`	Salto de línea. Sin él, todo sale en un mismo renglón.

printf es la función de C estándar para imprimir: aparece en cualquier libro de C. Le anteponemos **Serial.** para que el texto salga por el USB.

¿\n o \r\n? Cómo termina un renglón

\r — **Retorno de carro** (CR, código 13): vuelve al principio del renglón.

\n — **Salto de línea** (LF, código 10): baja un renglón.

Quién	Qué usa para terminar un renglón
Linux, macOS, C estándar	`\n`
Windows, protocolos de red	`\r\n`
Serial.println() de Arduino	`\r\n` (lo agrega solo)

✓ En el curso usamos **\n**: es C estándar y el monitor de PlatformIO y el de Wokwi lo muestran bien. Si una terminal muestra el texto **en escalera**, cambiar a **\r\n**.

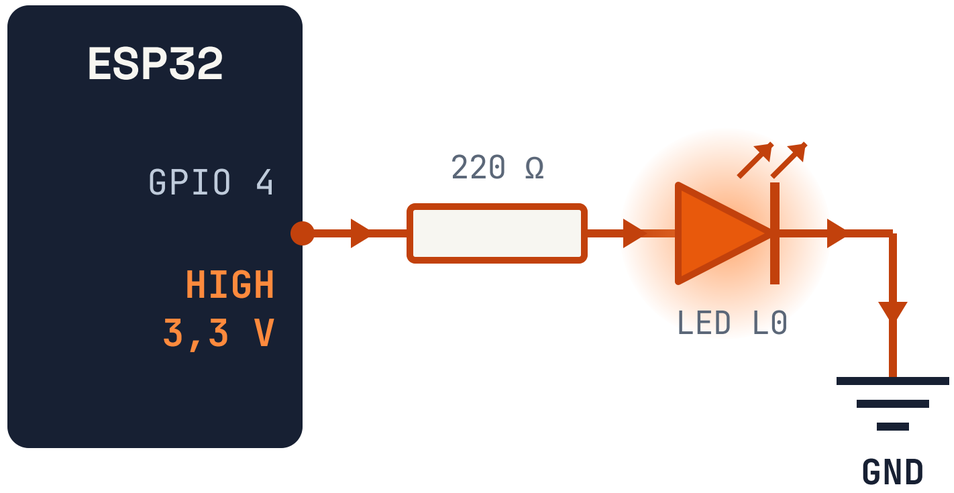
Salidas digitales: tres instrucciones nuevas

Instrucción	Qué hace
`pinMode(4, OUTPUT);`	Configura el pin 4 como **salida**. Va en setup().
`digitalWrite(4, HIGH);`	Pone el pin en **1** (3,3 V): el LED enciende.
`digitalWrite(4, LOW);`	Pone el pin en **0** (0 V): el LED apaga.
`delay(500);`	**Espera** 500 ms sin hacer nada.

HIGH y **LOW** son el 1 y el 0 de Sistemas Lógicos. Un pin de salida es una variable binaria que se ve con un LED.

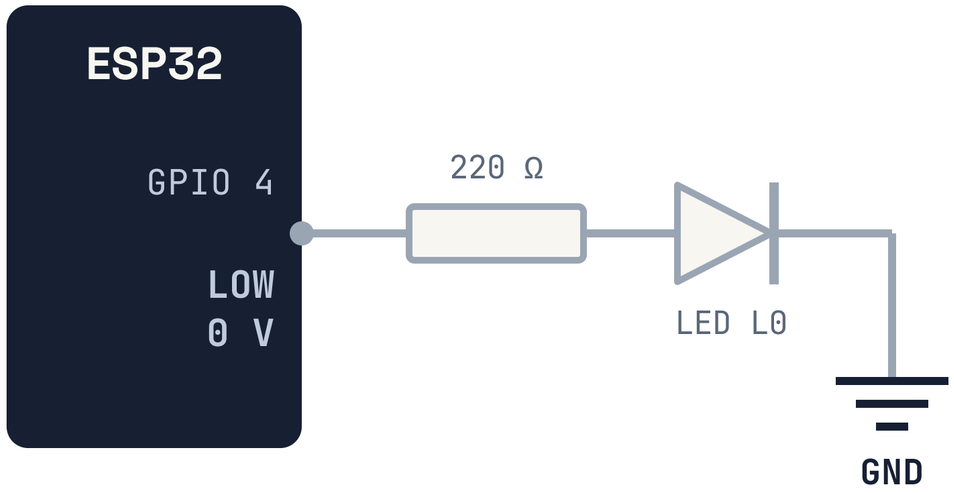
Así se conecta el LED L0

`digitalWrite(4, HIGH);`



LED encendido · circula corriente

`digitalWrite(4, LOW);`



LED apagado · no circula corriente

HIGH = 1 = 3,3 V = LED encendido. El Blink pasa de un dibujo al otro cada 500 ms.

El clásico Blink

```
void setup() {
  pinMode(4, OUTPUT);     // L0 es salida
}

void loop() {
  digitalWrite(4, HIGH);  // encender
  delay(500);             // esperar
  digitalWrite(4, LOW);   // apagar
  delay(500);             // esperar
}
```

Para pensar

¿Qué pasa si se borra el segundo **delay(500)**?

¿Por qué el LED parece estar siempre prendido?

Romper el Blink a propósito, de a un error

Romper esto	Mensaje típico
Borrar un punto y coma	`expected ';' before …`
Escribir **digitalwrite**	`'digitalwrite' was not declared in this scope`
Borrar una llave de cierre	`expected '}' at end of input`
Escribir **delay(500**	`expected ')' before ';' token`

Leer **el primer error**, no el último: uno causa otros diez.

El error está en la línea que indica **o en la anterior**.

6 · EJERCICIOS · 50 MIN

Ahora ustedes

EJERCICIO 1

Tocar los números

LED prendido 100 ms y apagado 900 ms. Después 50 y 50. ¿Hasta qué valor el ojo distingue el parpadeo?

EJERCICIO 2

SOS

L0 en Morse: tres cortos (200 ms), tres largos (600 ms), tres cortos y 2 s de pausa. Que imprima SOS.

EJERCICIO 3

Barrido

L0, L1, L2 y L3 de a uno, como el auto fantástico. Pines 4, 16, 17 y 18.

El ejercicio 3 queda largo y repetitivo. Está bien: en la Clase 4 lo hacemos en cinco líneas.

TP CLASE 1

Mi entrenador se presenta

1. Comentario de encabezado: nombre, clase y qué hace el programa.
2. Al arrancar, un cartel de presentación con tu nombre.
3. En loop(): encender L0, L1, L2 y L3 uno por uno, cada medio segundo, e imprimir cuál.
4. Apagar los cuatro, imprimir "Todos apagados" y esperar un segundo.

```
==============================
  Entrenador Logico de Ana
  Sistemas Logicos - UNSE 2026
==============================
```

Entrega: carpeta **SL2026-Apellido-Clase1** en ZIP, sin la carpeta .pio.

CHECKLIST DE SALIDA

Al terminar la clase, sé…

- [ ] explicar qué es un programa y qué hace el compilador
- [ ] decir la diferencia entre setup() y loop()
- [ ] imprimir un mensaje en el monitor serie
- [ ] encender y apagar un LED con pinMode y digitalWrite
- [ ] leer un error y encontrar la línea que lo causa

La semana que viene: hoy el micro solo repitió cosas. En la Clase 2 va a **recordar números y contar en binario**.

